UNIVERSITÀ DEGLI STUDI DI PADOVA
CORSO DI LAUREA IN INFORMATICA (L-31)

Corso di Ingegneria del Software
Anno Accademico 2025/2026

Specifica Tecnica

Gruppo: NightPRO

Sistema: SmartOrder

swe.nightpro@gmail.com

Data: 2026-03-31
Versione: 0.3

Tabella delle Versioni

Versione	Data	Autore/i	Descrizione delle Modifiche	Verificatore
0.3	2026-03-31	D. Biasuzzi, F. Zanella, S. Perozzo	Riorganizzazione documento, inserimento dei diagrammi PlantUML e stesura delle sezioni 4, 5, 6 e 7	M. M. Romascu
0.2	2026-03-26	Davide Biasuzzi	Inserimento paragrafo su esecuzione e su tecnologie utilizzate	Leonardo Bilato
0.1	2026-03-20	Davide Biasuzzi	Stesura base documento (Introduzione e Requisiti di Sistema)	Leonardo Bilato

Indice

Componenti del Gruppo 6

1 Introduzione 7
1.1 Scopo del documento . . . 7
1.2 Scopo del prodotto . . . 7
1.3 Glossario . . . 7
1.4 Riferimenti . . . 7
1.4.1 Riferimenti normativi . . . 7
1.4.2 Riferimenti informativi . . . 7

2 Ambiente minimo 8
2.1 Software minimo . . . 8
2.2 Hardware minimo . . . 8

3 Tecnologie utilizzate 9
3.1 Linguaggi di programmazione . . . 9
3.1.1 TypeScript . . . 9
3.1.2 Python . . . 9
3.1.3 SQL . . . 9
3.2 Framework e librerie . . . 9
3.2.1 Next.js . . . 9
3.2.2 React . . . 10
3.2.3 Tailwind CSS . . . 10
3.2.4 FastAPI . . . 10
3.2.5 Pydantic . . . 10
3.2.6 OpenAI SDK . . . 10
3.3 Strumenti di persistenza . . . 10
3.3.1 PostgreSQL . . . 10
3.3.2 psycopg2 . . . 11
3.4 Intelligenza artificiale . . . 11
3.4.1 GPT-4o . . . 11
3.4.2 GPT-4o-mini . . . 11
3.4.3 Whisper . . . 11
3.5 Infrastruttura e DevOps . . . 11
3.5.1 Docker . . . 11
3.5.2 Node.js . . . 11
3.5.3 Uvicorn . . . 12
3.5.4 GitHub Actions . . . 12
3.5.5 ESLint . . . 12

4 Architettura di sistema 13
4.1 Obiettivi architetturali . . . 13
4.2 Vista C4 - Livello 1 (System Context) . . . 13
4.3 Vista C4 - Livello 2 (Container) . . . 14
4.4 Stile architetturale adottato . . . 14

5 Architettura di deployment 16
5.1 Scenario di esecuzione . . . 16
5.2 Nodi e responsabilità di deployment . . . 16

5.3 Canali di comunicazione . . . 17
5.4 Aspetti operativi e CI/CD . . . 18

6 Design pattern **19**
6.1 Dependency Injection . . . 19
6.1.1 Descrizione del pattern . . . 19
6.1.2 Motivazioni dell'utilizzo del pattern . . . 19
6.1.3 Utilizzo del pattern nel progetto . . . 19
6.2 Adapter (Ports and Adapters) . . . 19
6.2.1 Descrizione del pattern . . . 19
6.2.2 Motivazioni dell'utilizzo del pattern . . . 19
6.2.3 Utilizzo del pattern nel progetto . . . 19
6.3 Repository Pattern . . . 20
6.3.1 Descrizione del pattern . . . 20
6.3.2 Motivazioni dell'utilizzo del pattern . . . 20
6.3.3 Utilizzo del pattern nel progetto . . . 20
6.4 Facade . . . 20
6.4.1 Descrizione del pattern . . . 20
6.4.2 Motivazioni dell'utilizzo del pattern . . . 20
6.4.3 Utilizzo del pattern nel progetto . . . 20
6.5 Observer (State Management Reattivo) . . . 20
6.5.1 Descrizione del pattern . . . 20
6.5.2 Motivazioni dell'utilizzo del pattern . . . 21
6.5.3 Utilizzo del pattern nel progetto . . . 21

7 Database **22**
7.1 Ruolo del database nel sistema . . . 22
7.2 Macro-aree informative . . . 22
7.3 Vincoli dati rilevanti . . . 23
7.4 Output verso ERP . . . 23

8 Architettura di dettaglio **24**
8.1 Frontend - C4 Livello 3 e 4 . . . 24
8.1.1 Layer 1 - Routing e Pages . . . 25
8.1.2 Layer 2 - State Management . . . 25
8.1.3 Layer 3 - Service/API Client . . . 26
8.1.4 Layer 4 - Real-time . . . 26
8.1.5 Layer 5 - UI Library . . . 26
8.2 Backend - C4 Livello 3 e 4 . . . 26
8.2.1 Controllers (Inbound Ports) . . . 28
8.2.2 Application Services (Use Cases) . . . 28
8.2.3 Core Domain . . . 28
8.2.4 Outbound Ports e Adapter . . . 29
8.3 Flusso applicativo principale (ordine) . . . 30

9 Sorgenti PlantUML **31**
9.1 Convenzioni operative . . . 33

Elenco delle figure

1 Diagramma C4 L1 (PlantUML compilato): contesto del sistema SmartOrder. . . 13

2 Diagramma C4 L2 (PlantUML compilato): vista container. 14
3 Vista di deployment logico del sistema. 16
4 Frontend L3 (PlantUML compilato): componenti principali. 24
5 Frontend L4 (PlantUML compilato): layered architecture orientata al coding. . . 25
6 Backend L3 (PlantUML compilato): componenti applicativi e dipendenze. 27
7 Backend L4 (PlantUML compilato): layer, ports e adapters. 28
8 Core Domain (PlantUML compilato): classi principali e relazioni. 29
9 Flusso checkout (PlantUML compilato): sequenza end-to-end. 30

Elenco delle tabelle

1 Componenti del gruppo NightPRO. 6
2 Software minimo per il deployment. 8
3 Hardware minimo richiesto. 8
4 C4 L1 - Contesto del sistema SmartOrder. 13
5 C4 L2 - Container e responsabilita. 14
6 Nodi logici di deployment e relative responsabilità. 17
7 Aree logiche del modello dati relazionale. 22
8 Elenco sorgenti PlantUML e output compilati dell'architettura. 32

Componenti del Gruppo

Cognome	**Nome**	**Matricola**
Biasuzzi	Davide	2111000
Bilato	Leonardo	2071084
Zanella	Francesco	2116442
Romascu	Mihaela-Mariana	2079726
Ogniben	Michele	2042325
Perozzo	Samuele	2110989
Ponso	Giovanni	2000558

Tabella 1: Componenti del gruppo NightPRO.

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di descrivere l'architettura software del sistema *SmartOrder*, definendo le scelte tecnologiche, i pattern architetturali e progettuali adottati, e l'architettura di dettaglio di ciascun componente. La Specifica Tecnica costituisce il riferimento principale per la fase di codifica, garantendo coerenza tra progettazione e implementazione.

1.2 Scopo del prodotto

SmartOrder è una piattaforma B2B per l'automazione della gestione degli ordini commerciali. Il sistema consente ai clienti di formulare ordini in linguaggio naturale (testo, audio, immagini) e li converte automaticamente in ordini strutturati pronti per l'integrazione con sistemi ERP aziendali, riducendo errori e tempi di inserimento manuale.

1.3 Glossario

Per evitare ambiguità, i termini tecnici e di dominio utilizzati nel documento sono definiti nel *Glossario v2.0*. I termini presenti nel glossario sono contrassegnati con il simbolo $_{\text{G}}$ alla prima occorrenza in ogni sezione.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- Norme di Progetto v2.0 (Ultima modifica 2026-03-20)

1.4.2 Riferimenti informativi

- Glossario v2.0 (Ultima modifica 2026-03-20)
- Piano di Qualifica v1.0 (Ultima modifica 2026-03-20)
- Next.js Documentation (Ultima modifica 2026-03-20)
- FastAPI Documentation (Ultima modifica 2026-03-20)
- OpenAI API Documentation (Ultima modifica 2026-03-20)
- PostgreSQL Documentation (Ultima modifica 2026-03-20)
- Software Architecture Patterns, SWE 2025-2026 (Ultima modifica 2026-03-26)
- Dependency Injection, SWE 2025-2026 (Ultima modifica 2026-03-26)
- Design Pattern Creazionali, SWE 2025-2026 (Ultima modifica 2026-03-26)
- Design Pattern Strutturali, SWE 2025-2026 (Ultima modifica 2026-03-26)

2 Ambiente minimo

2.1 Software minimo

Software	**Descrizione**	**Versione minima**
Node.js	Runtime JavaScript per l'esecuzione dell'applicazione	$\geq$ 20.0
npm	Gestore pacchetti Node.js	$\geq$ 10.0
Python	Interprete Python per il modulo AI	$\geq$ 3.11
pip	Gestore pacchetti Python	$\geq$ 23.0
PostgreSQL	Database relazionale	$\geq$ 16
Browser web	Chrome, Firefox, Safari, Edge	Ultime 2 versioni

Tabella 2: Software minimo per il deployment.

2.2 Hardware minimo

Componente	**Requisito**
CPU	Processore 64-bit, $\geq$ 2 core
RAM	$\geq$ 4 GB
Spazio disco	$\geq$ 5 GB
Connessione di rete	Richiesta per accesso alle API OpenAI

Tabella 3: Hardware minimo richiesto.

3 Tecnologie utilizzate

In questa sezione vengono elencate le tecnologie adottate per lo sviluppo del sistema *SmartOrder*. Per ciascuna tecnologia viene indicata la versione utilizzata, la motivazione della scelta e il contesto d'impiego nel progetto.

3.1 Linguaggi di programmazione

3.1.1 TypeScript

- **Versione:** 5
- **Documentazione ufficiale:** TypeScript Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Frontend
- **Contesto di utilizzo:** linguaggio principale per l'interfaccia utente dell'applicativo Next.js, sia per i componenti React che per la gestione dello stato lato client.
- **Vantaggi:** tipizzazione statica che riduce errori a runtime, ottimo supporto IDE, compatibilità nativa con React e Next.js.
- **Svantaggi:** overhead di compilazione rispetto a JavaScript puro.

3.1.2 Python

- **Versione:** 3.11
- **Documentazione ufficiale:** Python 3 Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** implementazione della pipeline di elaborazione degli ordini tramite intelligenza artificiale e gestione delle API REST tramite FastAPI.
- **Vantaggi:** ecosistema maturo in ambito NLP/ML, compatibilità nativa con le librerie OpenAI, sintassi concisa, supporto asincrono con Uvicorn/ASGI.
- **Svantaggi:** prestazioni inferiori a linguaggi compilati; mitigato dall'uso di Uvicorn (ASGI asincrono).

3.1.3 SQL

- **Documentazione ufficiale:** ISO/IEC 9075:2023 (SQL) (Ultima modifica 2026-03-26)
- **Ambito:** Database
- **Contesto di utilizzo:** interrogazioni sul catalogo prodotti, gestione ordini, sessioni e carrelli.

3.2 Framework e librerie

3.2.1 Next.js

- **Versione:** 15.5
- **Documentazione ufficiale:** Next.js Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Frontend full-stack
- **Contesto di utilizzo:** framework principale per il frontend. Gestisce il rendering dell'interfaccia utente (React) e la logica lato client.
- **Vantaggi:** App Router con Server Components, SSR e SSG, integrazione ottimale con React.
- **Svantaggi:** curva di apprendimento per le convenzioni App Router; vincolo a React come libreria UI.

3.2.2 React

- **Versione:** 19
- **Documentazione ufficiale:** React Documentation (Ultima modifica 2026-03-26)
- **Ambito:** UI
- **Contesto di utilizzo:** costruzione dell'interfaccia utente tramite componenti dichiarativi, gestione dello stato con Context API e Hooks.
- **Vantaggi:** ecosistema vastissimo, modello a componenti riutilizzabili, Virtual DOM per aggiornamenti efficienti.

3.2.3 Tailwind CSS

- **Versione:** 4.1
- **Documentazione ufficiale:** Tailwind CSS Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Styling
- **Contesto di utilizzo:** stilizzazione dei componenti React con approccio utility-first.
- **Vantaggi:** prototipazione rapida, design responsive$_G$ integrato, nessun CSS custom da mantenere.
- **Svantaggi:** classi verbose nel markup; mitigato dalla composizione in componenti React.

3.2.4 FastAPI

- **Versione:** 0.115
- **Documentazione ufficiale:** FastAPI Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** espone gli endpoint `/chat` e `/transcribe` per l'elaborazione AI, gestione degli ordini e delle sessioni. Funziona come backend principale dell'applicazione.
- **Vantaggi:** validazione automatica dei dati tramite Pydantic$_G$, documentazione OpenAPI auto-generata, supporto nativo async/await.
- **Svantaggi:** richiede un processo separato (Uvicorn) rispetto al runtime Node.js.

3.2.5 Pydantic

- **Versione:** 2.9
- **Documentazione ufficiale:** Pydantic Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** definizione dei modelli di request/response per le API FastAPI, con validazione automatica dei tipi.

3.2.6 OpenAI SDK

- **Versione:** 1.51
- **Documentazione ufficiale:** OpenAI API Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** SDK$_G$ ufficiale per l'integrazione con GPT-4o, GPT-4o-mini e Whisper.

3.3 Strumenti di persistenza

3.3.1 PostgreSQL

- **Versione:** 16
- **Documentazione ufficiale:** PostgreSQL 16 Documentation (Ultima modifica 2026-03-26)

- **Ambito:** Database
- **Contesto di utilizzo:** unica base dati del sistema, accessibile dal modulo AI tramite Python (`psycopg2`). Tutte le operazioni sul catalogo prodotti, ordini, sessioni e carrelli avvengono tramite FastAPI.
- **Vantaggi:** robustezza, supporto a query complesse e full-text search, estensibilità (pgvector per Vector Search$_G$).

3.3.2 psycopg2

- **Documentazione ufficiale:** psycopg2 Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Modulo AI / Backend
- **Contesto di utilizzo:** driver Python per PostgreSQL con connection pooling$_G$ (`SimpleConnectionPool`).

3.4 Intelligenza artificiale

3.4.1 GPT-4o

- **Versione:** 2024-08-06
- **Documentazione ufficiale:** OpenAI GPT-4o (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** modello linguistico principale per la comprensione degli ordini, la classificazione degli intenti e le decisioni di business. Utilizzato con temperature=0.1 per risposte deterministiche.

3.4.2 GPT-4o-mini

- **Documentazione ufficiale:** OpenAI GPT-4o-mini (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** modello leggero per l'estrazione rapida dei parametri di ricerca e la classificazione semantica degli intenti utente.

3.4.3 Whisper

- **Versione:** 1
- **Documentazione ufficiale:** OpenAI Speech-to-Text (Whisper) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** trascrizione di messaggi audio in testo, con prompt specifico per il dominio Ergon e lingua forzata a italiano.

3.5 Infrastruttura e DevOps

3.5.1 Docker

- **Versione:** $\geq$ 24.0
- **Documentazione ufficiale:** Docker Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** containerizzazione dell'intero sistema per garantire portabilità e riproducibilità degli ambienti. Docker Compose ($\geq$ 2.20) orchestra l'avvio dei container.

3.5.2 Node.js

- **Versione:** 20
- **Documentazione ufficiale:** Node.js 20 Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** runtime JavaScript per l'esecuzione di Next.js in sviluppo e produzione.

3.5.3 Uvicorn

- **Versione:** 0.32
- **Documentazione ufficiale:** Uvicorn Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** server ASGI$_G$ per l'esecuzione di FastAPI, con supporto a richieste asincrone.

3.5.4 GitHub Actions

- **Documentazione ufficiale:** GitHub Actions Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** pipeline di integrazione continua per quality checks (ESLint, build verification).

3.5.5 ESLint

- **Versione:** 9
- **Documentazione ufficiale:** ESLint Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** linter$_G$ statico per il codice TypeScript/JavaScript.

4 Architettura di sistema

Questa sezione illustra l'architettura ad alto livello del sistema SmartOrder. Vengono definiti gli obiettivi progettuali che hanno guidato le scelte tecnologiche e viene fornita una rappresentazione progressiva del sistema utilizzando il modello C4, partendo dal contesto generale fino ad arrivare ai container principali. Infine, viene motivato lo stile architetturale adottato.

4.1 Obiettivi architetturali

L'architettura del sistema *SmartOrder* e stata definita per soddisfare i seguenti obiettivi:

- separare in modo netto responsabilita di presentazione, logica applicativa e persistenza;
- mantenere il backend indipendente dalla tecnologia client;
- supportare il flusso *Human-in-the-Loop* per i casi a bassa affidabilita AI;
- garantire integrazione con ERP tramite output JSON su cartella condivisa;
- favorire estendibilita verso nuovi canali di input (es. immagini, canali esterni).

4.2 Vista C4 - Livello 1 (System Context)

Il diagramma C4 Livello 1 (pagina `C4L1` del file `C4_SmartOrder.drawio`) individua i confini del sistema e gli attori esterni principali.

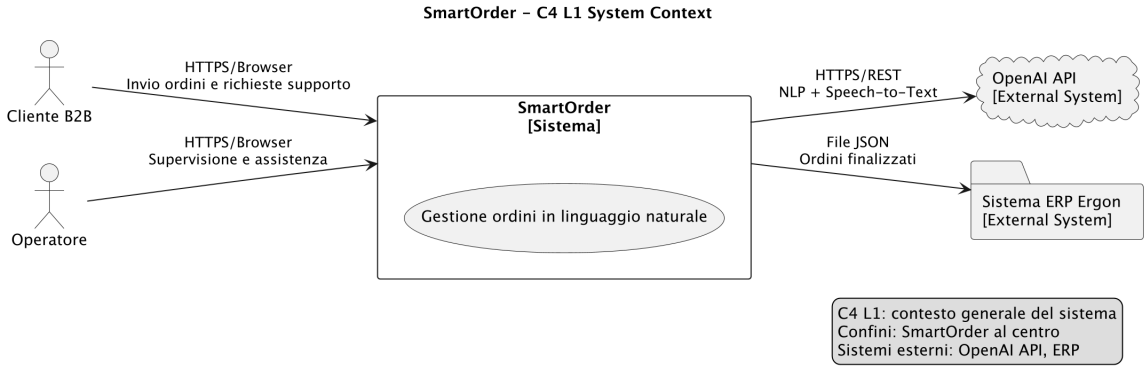

Figura 1: Diagramma C4 L1 (PlantUML compilato): contesto del sistema SmartOrder.

Elemento	Ruolo nel contesto
Cliente	Agente commerciale B2B che inserisce ordini in linguaggio naturale e richiede assistenza.
Operatore	Utente interno che supervisiona ticket, casi a bassa confidence e correzioni AI.
SmartOrder	Sistema oggetto della specifica; acquisisce input multimodale e genera ordini strutturati.
OpenAI API	Servizio esterno usato per comprensione linguaggio naturale e trascrizione audio.
Sistema ERP	Gestionale aziendale che riceve ordini finalizzati in formato JSON.

Tabella 4: C4 L1 - Contesto del sistema SmartOrder.

4.3 Vista C4 - Livello 2 (Container)

Il diagramma C4 Livello 2 (pagina `C4L2`) decompone SmartOrder nei container applicativi principali.

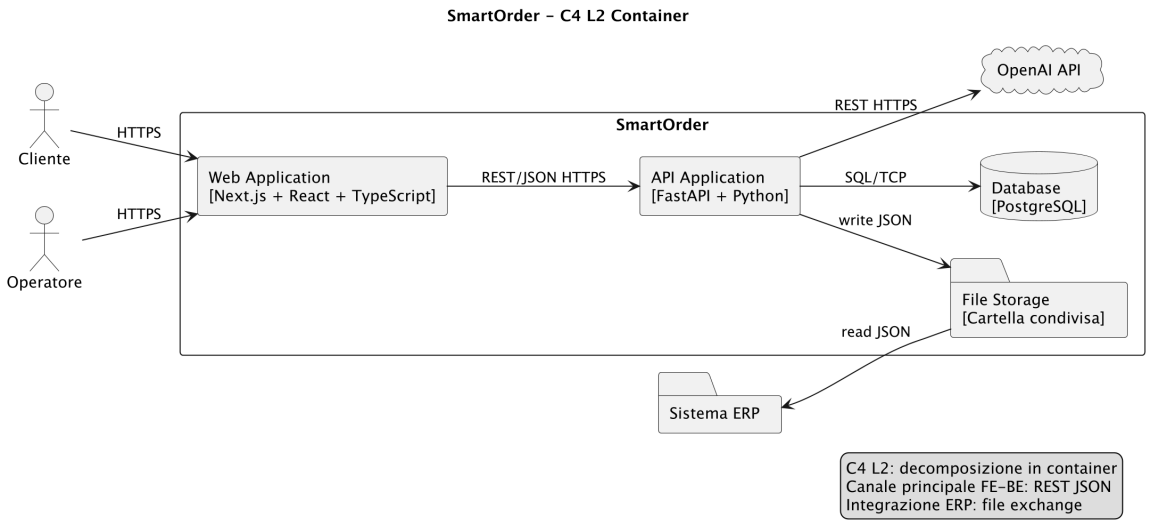

Figura 2: Diagramma C4 L2 (PlantUML compilato): vista container.

Container	**Responsabilita**
Web Application (Next.js + React + TypeScript)	Interfaccia cliente/operatore, gestione sessione utente, rendering chat, carrello, storico, dashboard operative.
API Application (FastAPI)	Business logic di dominio: autenticazione, orchestrazione AI, gestione carrello/ordini/ticket, persistenza e pubblicazione output.
Database (PostgreSQL)	Persistenza strutturata di utenti, sessioni, carrelli, ordini, ticket, log AI e catalogo.
File Storage (cartella condivisa)	Punto di consegna ordini finalizzati in JSON verso processo ERP esterno.

Tabella 5: C4 L2 - Container e responsabilita.

4.4 Stile architetturale adottato

La soluzione adotta uno stile **client-server a servizi**, con frontend e backend separati e API REST come contratto principale. All'interno del backend viene applicata una suddivisione per layer logici (Controller, Application Service, Core Domain, Outbound Ports) coerente con i principi di *Ports and Adapters*.

Le principali motivazioni della scelta sono:

- isolamento della logica di business da dipendenze infrastrutturali;
- testabilita delle regole applicative in assenza di interfaccia grafica;

- sostituibilita di adapter esterni (servizi AI, storage, connettori DB) con impatto ridotto;
- evoluzione indipendente dei rilasci frontend e backend.

5 Architettura di deployment

Questa sezione descrive la distribuzione fisica dei componenti del sistema *SmartOrder*, illustrando gli ambienti di esecuzione, i nodi infrastrutturali coinvolti e le strategie di Continuous Integration e Continuous Deployment (CI/CD) adottate.

5.1 Scenario di esecuzione

Il sistema prevede due configurazioni principali per l'esecuzione, pensate per separare l'ambiente di sviluppo da quello di produzione:

- **Ambiente di Sviluppo (Locale)**: i servizi di frontend e backend vengono eseguiti localmente sulla macchina di sviluppo, sfruttando la containerizzazione (es. Docker Compose) o task eseguiti in locale per avviare l'applicativo e un database PostgreSQL isolato. Questo garantisce un ambiente di test riproducibile.
- **Ambiente di Produzione (Cloud)**: l'intero applicativo è ospitato sulla piattaforma cloud **Railway**. Railway gestisce nativamente il provisioning delle risorse, ospitando in container isolati i servizi di frontend (Next.js), backend (FastAPI) e il database relazionale (PostgreSQL).

5.2 Nodi e responsabilità di deployment

La seguente vista descrive la mappatura tra i container software e le risorse cloud che li ospitano in ambiente di produzione.

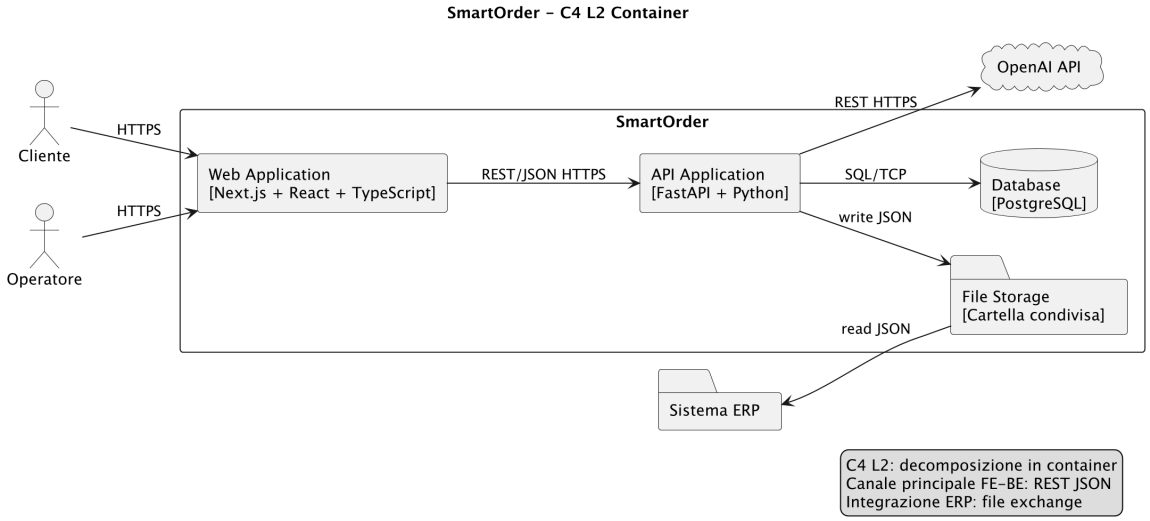

Figura 3: Vista di deployment logico del sistema.

Nodo Cloud	Responsabilità
Frontend Service(Railway)	Espone la web application in Next.js tramite protocollo HTTPS. Gestisce il rendering dell'interfaccia utente, l'idratazione lato client e inoltra le richieste API verso il backend.
Backend Service(Railway)	Esegue il server Uvicorn con l'applicazione FastAPI. Centralizza la logica di business, l'orchestrazione delle chiamate AI e la validazione transazionale degli ordini.
Database Service(Railway PostgreSQL)	Istanza cloud gestita di PostgreSQL. Conserva i dati transazionali (utenti, ordini, carrelli, ticket) garantendo integrità, persistenza e backup continui.
File Storage Condiviso(Object Storage S3)	Bucket cloud (es. Amazon S3 o compatibile) utilizzato come buffer di integrazione aziendale. Il backend vi deposita i file JSON degli ordini finalizzati, i quali vengono successivamente prelevati dal sistema ERP esterno.
OpenAI Cloud API(Servizio Esterno)	Endpoint gestiti direttamente da OpenAI, richiamati dal backend per i servizi di Natural Language Processing (GPT-4o) e di speech-to-text (Whisper).

Tabella 6: Nodi logici di deployment e relative responsabilità.

Nota architetturale sullo Storage Condiviso: Per l'integrazione con l'ERP aziendale si è scelto di adottare un **Object Storage**. Rispetto a una semplice cartella di rete, un bucket cloud offre standard di sicurezza enterprise, garantisce isolamento dei permessi e fornisce un facile accesso tramite API per il polling da parte di sistemi terzi.

5.3 Canali di comunicazione

Le comunicazioni tra i vari nodi del sistema avvengono esclusivamente tramite protocolli sicuri e standardizzati:

- **Frontend ↔ Backend**: le chiamate API avvengono su canale cifrato HTTPS, scambiando payload in formato JSON. L'autenticazione è garantita tramite token o cookie di sessione sicuri.
- **Backend ↔ PostgreSQL**: connessione su protocollo TCP proprietario del database, ottimizzata tramite un meccanismo di *connection pooling* (gestito tramite `psycopg2`) per sopportare picchi di traffico.
- **Backend ↔ OpenAI API**: invocazioni REST su rete pubblica cifrata HTTPS verso i server cloud di OpenAI.
- **Backend → File Storage**: scritture nel bucket cloud tramite chiamate API autenticate, necessarie a depositare l'ordine json per il sistema ERP.

5.4 Aspetti operativi e CI/CD

Per garantire rilasci continui, controllati e privi di disservizi, il progetto adotta pratiche automatizzate di *Continuous Integration* e *Continuous Deployment* (CI/CD):

- **Continuous Integration (CI)**: grazie a *GitHub Actions*, all'apertura di ogni Pull Request o push sui rami principali viene triggerato un workflow di controllo qualità (es. linter tramite ESLint) per prevenire il rilascio di codice formalmente scorretto.
- **Continuous Deployment (CD) automatizzato**: il deployment in produzione è delegato a **Railway**, il quale è interfacciato direttamente al repository GitHub. Ad ogni push sul branch principale, Railway rileva automaticamente i cambiamenti e lancia il processo di build e deploy. Questo avviene in modalità *zero-downtime*, aggiornando i container senza causare interruzioni agli utenti.
- **Gestione dei Segreti**: chiavi API, credenziali del database e secret di cifratura non vengono mai committati nel repository. Sono configurati come variabili d'ambiente (Environment Variables) criptate direttamente all'interno della dashboard del progetto su Railway.

6 Design pattern

Questa sezione descrive i principali pattern architetturali e progettuali (ispirati alle soluzioni della *Gang of Four* e ai pattern enterprise) adottati per risolvere problemi ricorrenti nel sistema *SmartOrder*, sia lato backend che lato frontend.

6.1 Dependency Injection

6.1.1 Descrizione del pattern

La Dependency Injection è un pattern strutturale che implementa l'Inversione del Controllo (IoC). Consiste nel fornire le dipendenze (oggetti o servizi) a una classe dall'esterno, anziché lasciare che sia la classe stessa a istanziarle internamente.

6.1.2 Motivazioni dell'utilizzo del pattern

L'obiettivo principale è il disaccoppiamento dei componenti. Iniettando le dipendenze si rispetta il principio di Inversione delle Dipendenze (la "D" dei principi SOLID), rendendo il codice estremamente più modulare. Questo risulta fondamentale per la testabilità del sistema: permette infatti di iniettare agilmente dei *mock* o *stub* al posto dei servizi reali durante l'esecuzione degli unit test.

6.1.3 Utilizzo del pattern nel progetto

Il pattern è applicato in modo sistematico nel backend FastAPI. I controller del Livello 1 (Inbound Ports), come il `ConversationController` o il `CartController`, non creano mai direttamente le istanze dei service. Al contrario, ricevono i servizi necessari (es. `ConversationService`, `CartService`) tramite il meccanismo nativo di iniezione delle dipendenze del framework.

6.2 Adapter (Ports and Adapters)

6.2.1 Descrizione del pattern

L'Adapter è un pattern strutturale che permette a due interfacce incompatibili di collaborare. Agisce come un "traduttore" o un "ponte", incapsulando la logica di un sistema esterno e offrendo un'interfaccia compatibile con il dominio interno dell'applicazione.

6.2.2 Motivazioni dell'utilizzo del pattern

In un'architettura di tipo *Ports and Adapters* (Architettura Esagonale), questo pattern è cruciale per isolare la logica di business (il "Core") dalle dipendenze tecnologiche esterne. Evita che i dettagli di librerie di terze parti o SDK "inquinino" il dominio applicativo, permettendo di sostituire un fornitore di servizi in futuro modificando solo l'Adapter, senza toccare le regole di business.

6.2.3 Utilizzo del pattern nel progetto

Nel backend, il pattern si concretizza nel Livello 5 (Infrastructure Adapters). Interfacce come `IAIClient` fungono da porte in uscita (Outbound Ports). La classe `OpenAIAdapter` implementa questa interfaccia e agisce come Adapter verso le API esterne, traducendo le entità del dominio di SmartOrder in formato JSON/REST specifico richiesto dai server di OpenAI, proteggendo il resto del sistema dai dettagli della libreria esterna.

6.3 Repository Pattern

6.3.1 Descrizione del pattern

Il Repository è un pattern architetturale (di livello enterprise) che media tra la logica di dominio e il layer di data mapping. Fornisce un'interfaccia che simula una collezione di oggetti in memoria, nascondendo le complessità legate all'estrazione e al salvataggio dei dati persistenti.

6.3.2 Motivazioni dell'utilizzo del pattern

L'utilizzo del Repository centralizza l'accesso al database, evitando la frammentazione delle query SQL all'interno della logica di business. Garantisce un singolo punto di verità per le operazioni CRUD, semplifica la manutenzione del codice legato alla persistenza e facilita eventuali migrazioni tecnologiche.

6.3.3 Utilizzo del pattern nel progetto

Nel backend, l'interazione con il database relazionale (PostgreSQL) è astratta attraverso le interfacce definite nel Livello 4 (Outbound Ports) come `IUserRepository`, `IOrderRepository` e `ITicketRepository`. L'implementazione pratica di queste interfacce è delegata all'infrastruttura sottostante (come `PostgresAdapter` o `IDBSessionManager`), in modo che i servizi applicativi lavorino unicamente con oggetti di dominio.

6.4 Facade

6.4.1 Descrizione del pattern

Il Facade è un pattern strutturale che fornisce un'interfaccia unificata, astratta e semplificata a un insieme di interfacce o moduli più complessi all'interno di un sottosistema, nascondendone i dettagli di basso livello.

6.4.2 Motivazioni dell'utilizzo del pattern

Il pattern riduce l'accoppiamento tra i client e il sottosistema complesso, offrendo un punto di accesso centralizzato e facile da usare. Questo migliora la leggibilità del codice, agevola lo sviluppo e riduce la curva di apprendimento per i componenti che devono interfacciarsi con logiche articolate.

6.4.3 Utilizzo del pattern nel progetto

Nel frontend React/Next.js, i moduli presenti nel Livello 3 (Service Layer API Client), come `chatService`, `cartService` e `ticketService`, fungono da Facade verso la rete. Nascondono ai componenti visivi React l'intera complessità delle comunicazioni HTTP/REST verso FastAPI (iniezione dei token JWT negli header, gestione errori e serializzazione JSON), offrendo un'interfaccia pulita e semplice da invocare.

6.5 Observer (State Management Reattivo)

6.5.1 Descrizione del pattern

L'Observer è un pattern comportamentale che definisce una dipendenza uno-a-molti tra oggetti. Quando l'oggetto osservato (Subject) cambia stato, tutti gli oggetti dipendenti (Observer) vengono notificati e aggiornati automaticamente.

6.5.2 Motivazioni dell'utilizzo del pattern

Nel contesto dello sviluppo di interfacce utente, l'Observer è vitale per garantire che la UI rifletta sempre lo stato attuale dei dati senza dover implementare costosi meccanismi di polling. Permette di creare interfacce dinamiche e reattive in cui i componenti si limitano ad "ascoltare" le modifiche pertinenti.

6.5.3 Utilizzo del pattern nel progetto

Applicato nel frontend Next.js per la gestione dello stato globale al Livello 2. I vari Context Provider (es. `ChatContext`, `CartContext`, `NotificationContext`) assumono il ruolo di Subject, mantenendo la verità sui dati correnti. Event handler come `Chat Event Handler` o `Ticket Event Handler` ascoltano il *RT Gateway* tramite WebSocket e aggiornano i contesti. I componenti grafici dell'interfaccia iscritti (Observer) vengono così notificati, innescando un re-render istantaneo della sola porzione interessata.

7 Database

Questa sezione descrive l'architettura dei dati del sistema *SmartOrder*, definendo il ruolo del database, le macro-aree informative gestite, i vincoli di integrità applicati e le modalità di interscambio dati con il sistema gestionale esterno.

7.1 Ruolo del database nel sistema

Il sistema si avvale di **PostgreSQL** (in hosting cloud su Railway) come unica fonte dati transazionale (Single Source of Truth) per il backend FastAPI. Il database relazionale assolve a tre ruoli fondamentali:

- **Persistenza Transazionale:** memorizza in modo consistente lo stato delle sessioni, dei carrelli in lavorazione, dei ticket di assistenza e lo storico degli ordini consolidati.
- **Replica del Catalogo ERP:** ospita le tabelle anagrafiche dei prodotti e dei clienti sincronizzate dal gestionale aziendale, permettendo al sistema di operare con logiche di business locali (es. verifica assortimenti).
- **Motore di Ricerca Vettoriale:** tramite l'estensione `pgvector`, il database archivia gli *embeddings* generati dai modelli LLM e risolve le query semantiche di "Vector Search", mappando le descrizioni in linguaggio naturale ai codici articolo reali.

7.2 Macro-aree informative

Il modello relazionale del database è logicamente suddiviso in macro-aree, ciascuna responsabile di un dominio specifico dell'applicazione.

Area Logica	Contenuto principale e Responsabilità
Identità e Accesso	Entità relative agli utenti, ruoli (Cliente, Operatore), credenziali crittografate (hash scrypt) e gestione dei token/sessioni attive.
Catalogo e Assortimento	Anagrafica articoli (`anaart`), metadati descrittivi, *embeddings* vettoriali per la ricerca semantica e tabelle di visibilità/assortimento per singolo cliente (`asscli`).
Carrello e Ordini	Record transazionali che rappresentano il carrello corrente della sessione, le singole righe prodotto con le relative quantità e lo storico degli ordini finalizzati pronti per la consultazione (Storico Ordini).
Ticketing e Chat	Entità per la gestione dell'assistenza (*Human-in-the-loop*). Include i record dei Ticket (stati: "Aperto", "In lavorazione", "Chiuso"), l'assegnazione all'Operatore (Lock) e il tracciato cronologico (Log) dei messaggi di chat scambiati.
Log AI e Ottimizzazione	Archiviazione diagnostica delle interazioni con il modello LLM. Comprende il *Confidence Score*, i feedback degli utenti e le discrepanze generate dalle correzioni manuali ai carrelli, utili al *retraining*.

Tabella 7: Aree logiche del modello dati relazionale.

7.3 Vincoli dati rilevanti

Per garantire la robustezza del sistema e prevenire l'invio di ordini commercialmente invalidi al sistema ERP, il database impone rigorosi vincoli di integrità e di dominio:

- **Vincolo di Assortimento (Foreign Key logica):** un prodotto può essere inserito nel carrello di una sessione solo se esiste una corrispondenza valida nella tabella degli assortimenti autorizzati (`asscli`) per il Cliente in esame.
- **Vincolo di Disponibilità:** controlli a livello database (`anaart`) impediscono l'immissione in ordine di articoli marcati con stato "sospeso" o "non disponibile".
- **Integrità Referenziale Standard:** chiavi esterne (Foreign Keys) con regole di propagazione *ON DELETE CASCADE/RESTRICT* collegano indissolubilmente le righe dell'ordine all'ID Ordine padre e alla relativa anagrafica Cliente, prevenendo record orfani.
- **Vincolo di Quantità Commerciale:** i campi quantitativi delle righe ordine possiedono vincoli (*CHECK constraints*) per impedire valori nulli, negativi o incompatibili con i multipli di vendita (es. logica a "casse").

7.4 Output verso ERP

Il database di SmartOrder funge da ambiente di *staging* e composizione. Al termine del processo di checkout, i dati transazionali (intestazione ordine, righe, quantità e ID cliente) estratti dalle tabelle relazionali subiscono un processo di serializzazione.

Il sistema genera un **payload strutturato in formato JSON** conforme al contratto di integrazione concordato con il fornitore del gestionale. Questo file non viene immagazzinato a lungo termine nel database cloud, ma viene immediatamente depositato nel **File Storage condiviso (Bucket S3)**, che rappresenta il reale punto di aggancio e consegna formale della transazione verso il sistema ERP aziendale.

8 Architettura di dettaglio

8.1 Frontend - C4 Livello 3 e 4

Le pagine `C4L3F` e `C4L4F` descrivono una struttura a layer del frontend Next.js.

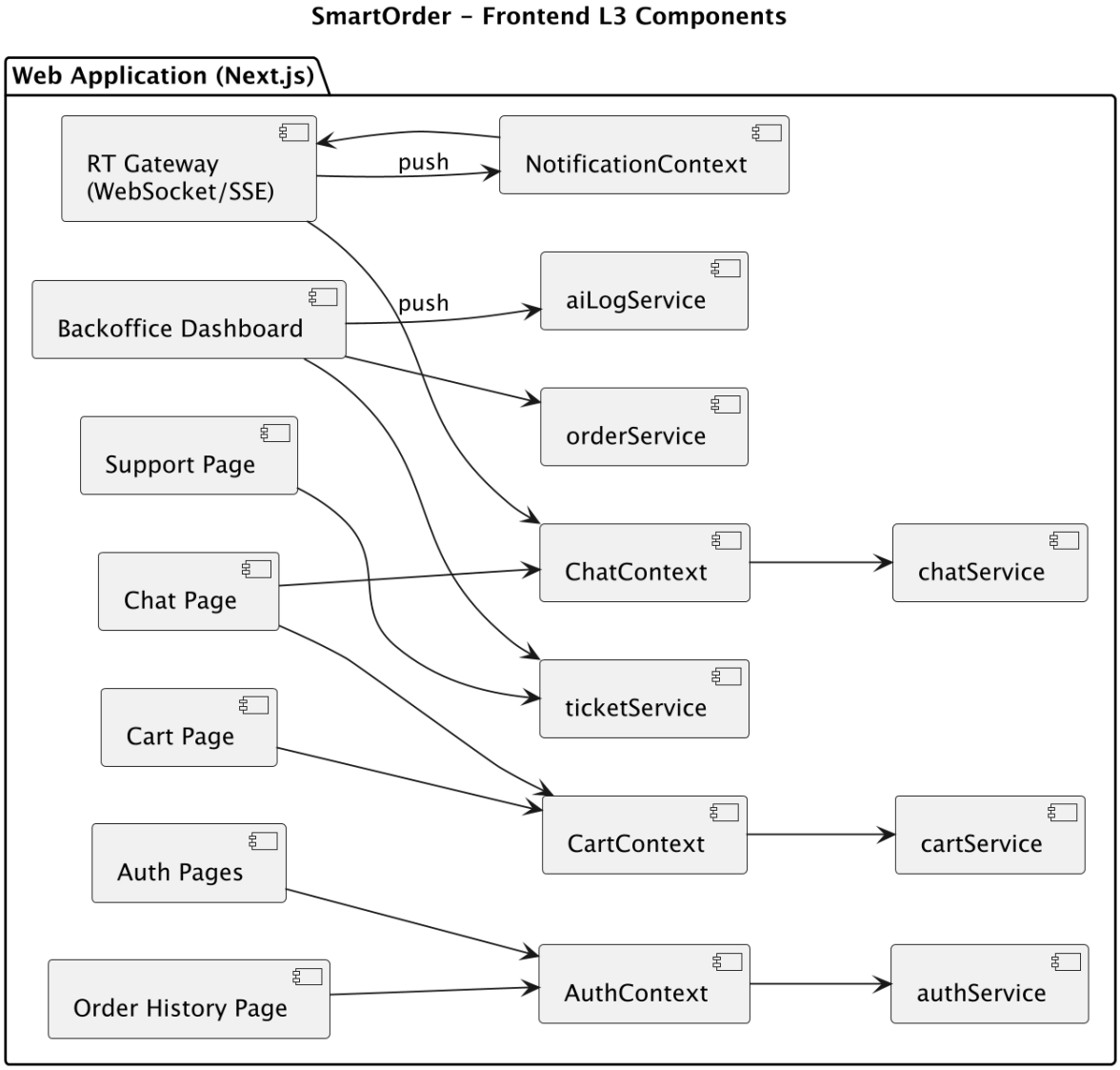

Figura 4: Frontend L3 (PlantUML compilato): componenti principali.

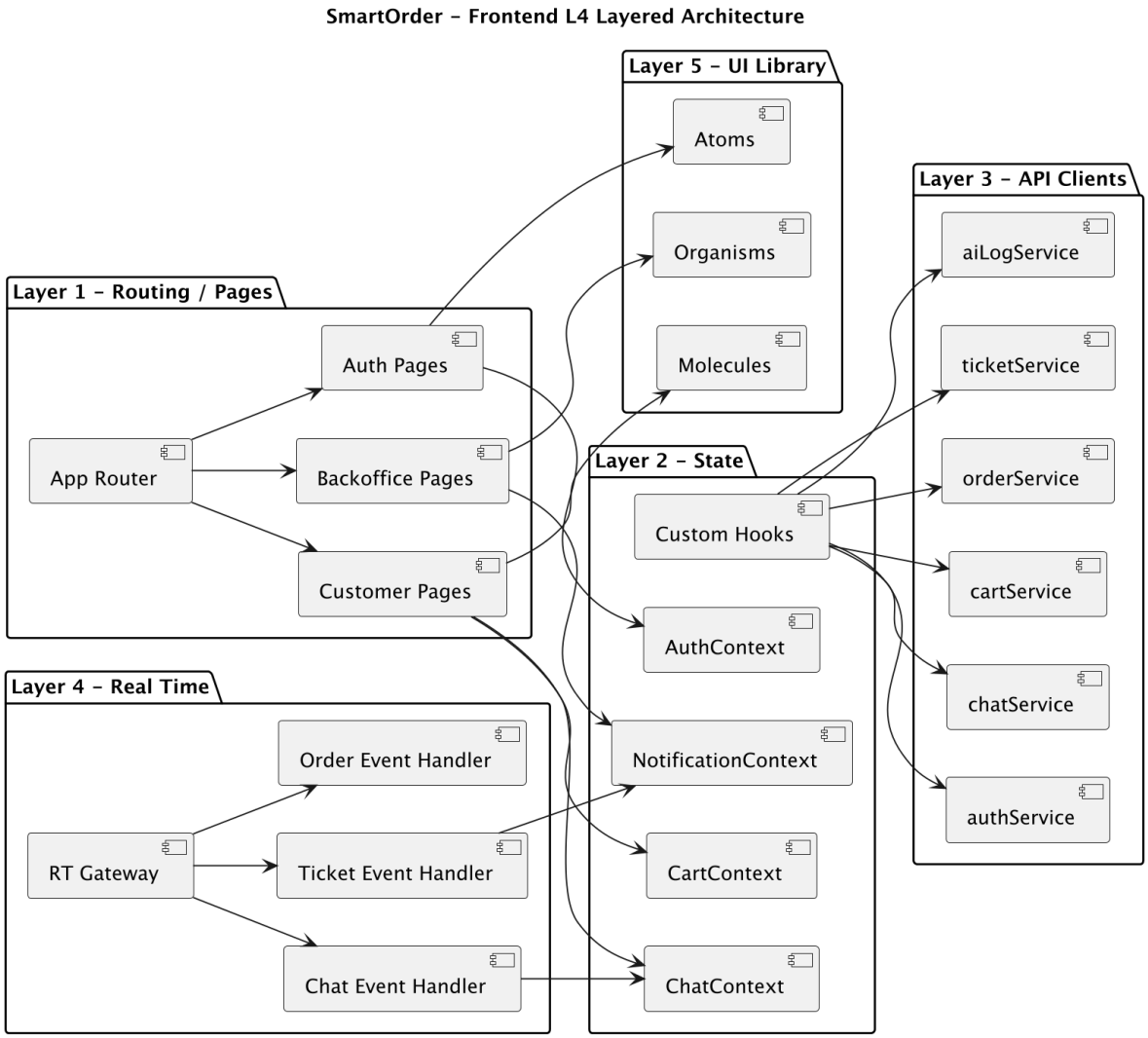

Figura 5: Frontend L4 (PlantUML compilato): layered architecture orientata al coding.

8.1.1 Layer 1 - Routing e Pages

Organizza route e viste principali:

- area autenticazione (login, recupero password, profilo);
- area cliente (chat, carrello, storico, supporto);
- area operatore/backoffice (ticket, ordini globali, dashboard AI).

8.1.2 Layer 2 - State Management

Utilizza Context e custom hooks per stato condiviso:

- `AuthContext`: identita e ruolo;
- `CartContext`: stato carrello e confidence score;
- `ChatContext`: messaggi live e modalita AI/Operatore;
- `NotificationContext`: eventi e notifiche applicative.

8.1.3 Layer 3 - Service/API Client

Ogni dominio espone un modulo dedicato (auth, cart, product, order, ticket, chat, ai-log) con responsabilita circoscritte e contratto REST verso backend.

8.1.4 Layer 4 - Real-time

Un gateway applicativo gestisce WebSocket/SSE e aggiorna in push chat, ticket, ordini e log AI.

8.1.5 Layer 5 - UI Library

Componenti organizzati secondo *Atomic Design* (atoms, molecules, organisms) per uniformare esperienza utente tra aree cliente e operatore.

8.2 Backend - C4 Livello 3 e 4

Le pagine `C4L3B` e `C4L4B` rappresentano la decomposizione interna del container FastAPI.

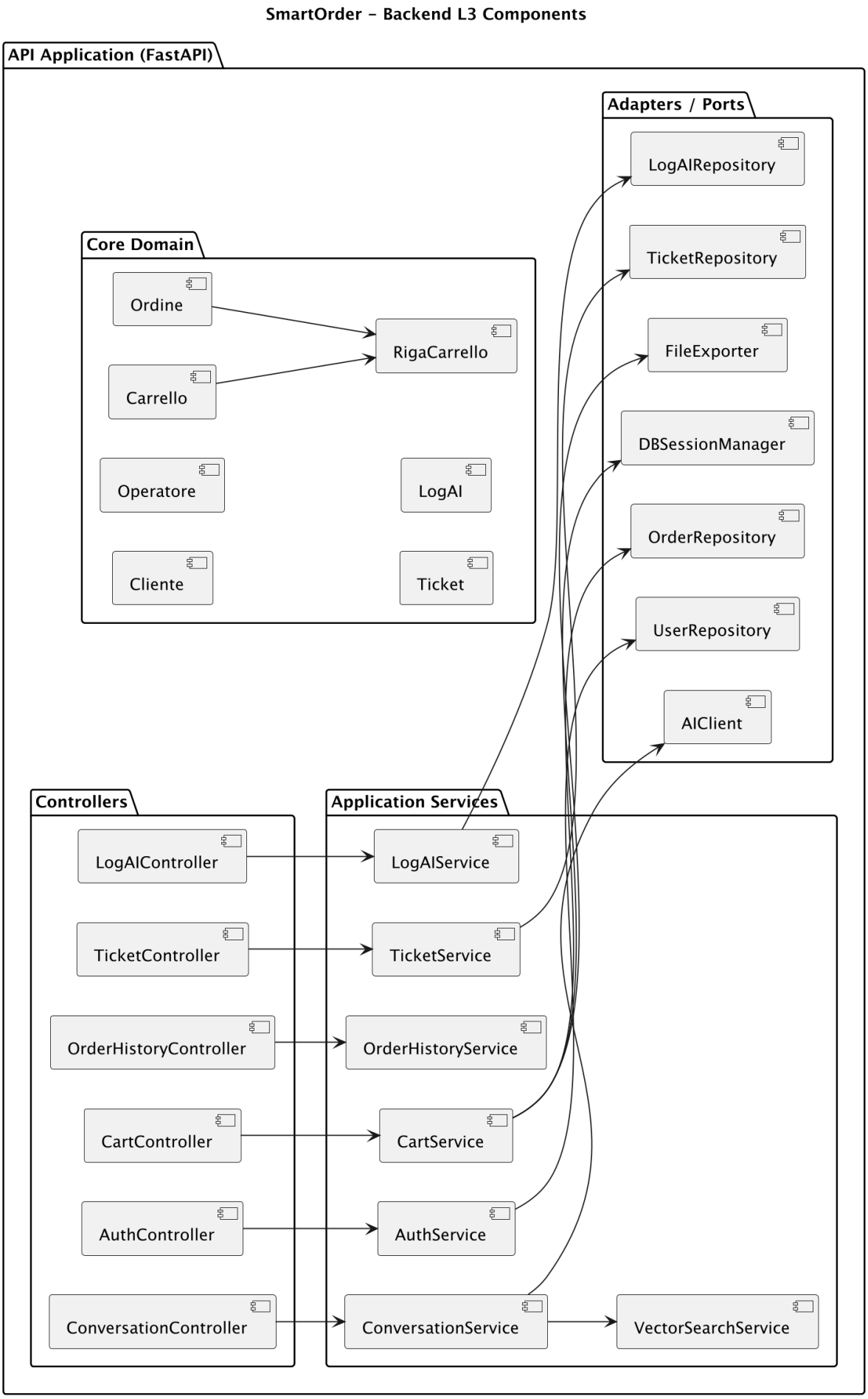

Figura 6: Backend L3 (PlantUML compilato): componenti applicativi e dipendenze.

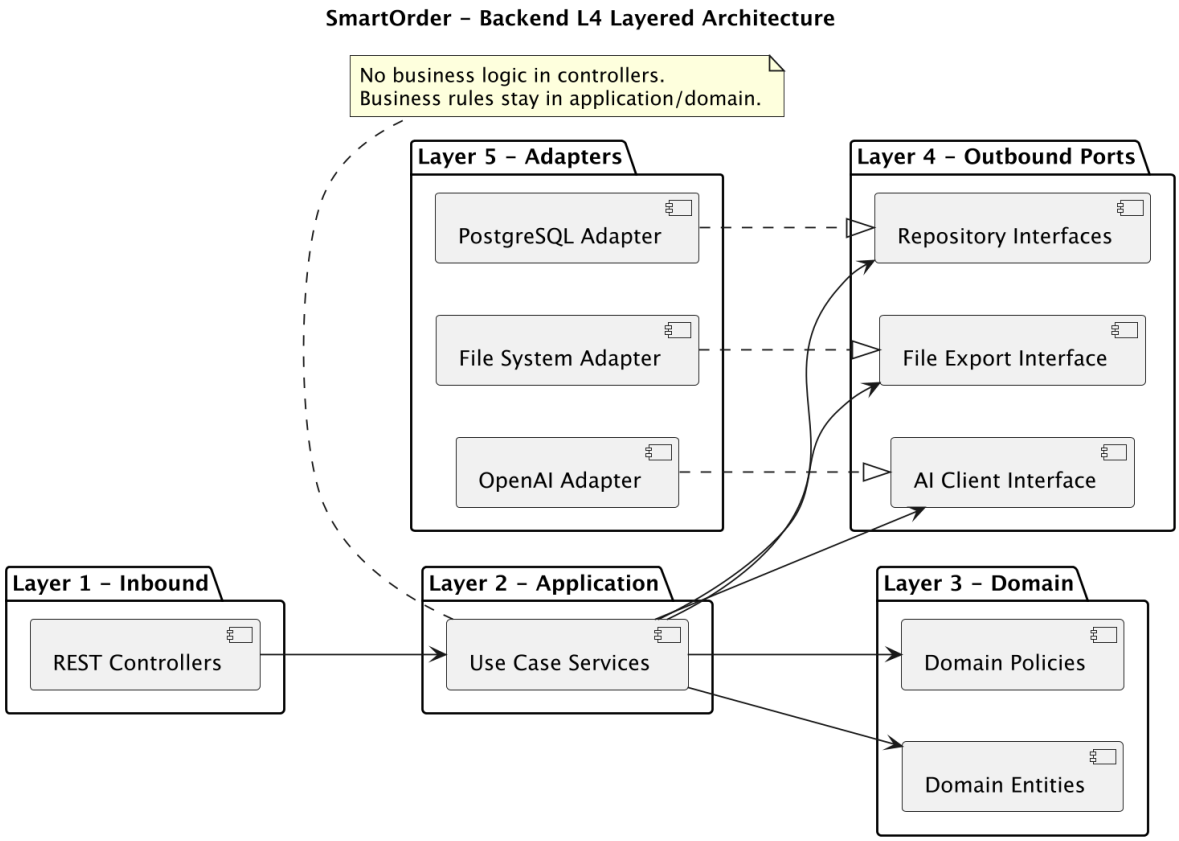

Figura 7: Backend L4 (PlantUML compilato): layer, ports e adapters.

8.2.1 Controllers (Inbound Ports)

Punto di ingresso HTTP per i principali domini: `AuthController`, `ConversationController`, `CartController`, `OrderHistoryController`, `TicketController`, `LogAIController`.

8.2.2 Application Services (Use Cases)

Orchestrano regole e collaborazioni tra moduli: `AuthService`, `ConversationService`, `CartService`, `OrderHistoryService`, `VectorSearchService`, `TicketService`, `LogAIService`.

8.2.3 Core Domain

Entita principali del dominio applicativo: Cliente, Operatore, Carrello, RigaCarrello, Ordine, Ticket, LogAI.

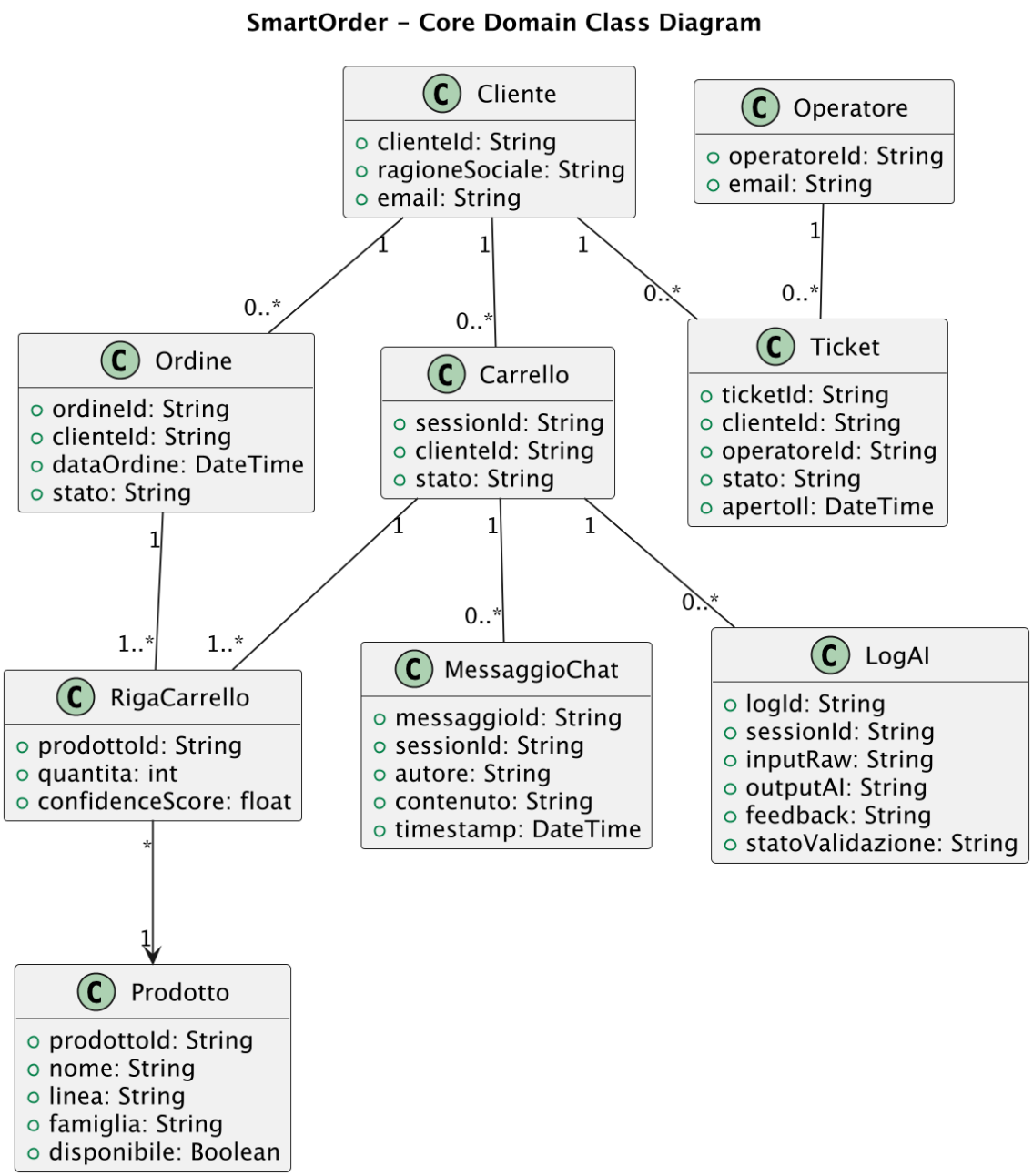

Figura 8: Core Domain (PlantUML compilato): classi principali e relazioni.

8.2.4 Outbound Ports e Adapter

Interfacce astratte per dipendenze esterne:

- repository dati (utenti, ordini, ticket, log);
- manager sessioni DB;
- client AI verso OpenAI;
- exporter file JSON per integrazione ERP.

8.3 Flusso applicativo principale (ordine)

1. Il Cliente invia input naturale (testo/audio/immagine) dalla Chat Page.
2. Il backend interpreta input e aggiorna il carrello con confidence score associato.
3. Il Cliente (o Operatore in delega) conferma il checkout del carrello.
4. Il backend valida vincoli (assortimento/disponibilita), persiste l'ordine e genera JSON.
5. Il file JSON viene depositato in storage condiviso per acquisizione ERP.

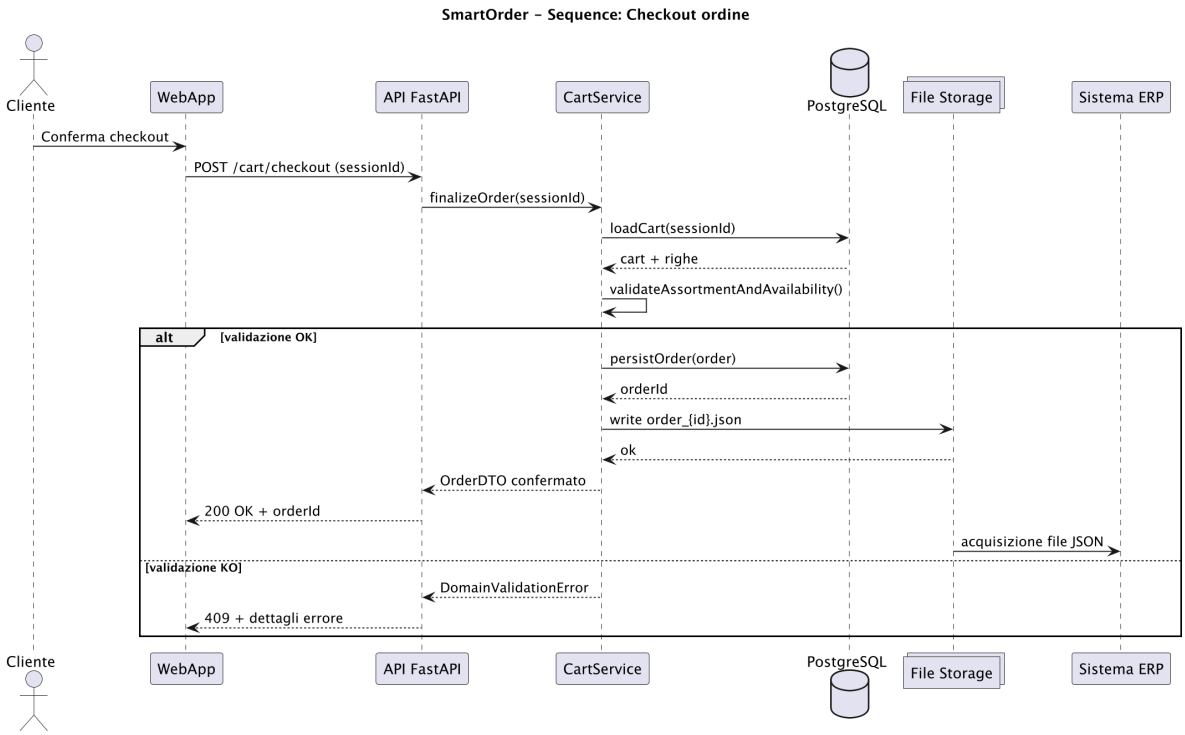

Figura 9: Flusso checkout (PlantUML compilato): sequenza end-to-end.

9 Sorgenti PlantUML

Questa sezione elenca i sorgenti `.puml` ufficiali da usare per generare i diagrammi architetturali. Per ogni sorgente viene mantenuto anche l'output compilato in formato `.png`, cosi da poter includere direttamente le figure nel documento. I file sono mantenuti nella directory `Contenuti/` e versionati insieme alla specifica.

File .puml	**Output .png**	**Contenuto**
`arch_c4_l1_context.puml`	`arch_c4_l1_context.png`	C4 Livello 1: attori esterni, sistema SmartOrder, sistemi terzi.
`arch_c4_l2_container.puml`	`arch_c4_l2_container.png`	C4 Livello 2: WebApp, API, DB, File Storage, OpenAI, ERP.
`arch_frontend_l3_components.puml`	`arch_frontend_l3_components.png`	C4 Livello 3 frontend: pagine, context, servizi API client, real-time.
`arch_backend_l3_components.puml`	`arch_backend_l3_components.png`	C4 Livello 3 backend: controller, service, core, adapter.
`arch_frontend_l4_layers.puml`	`arch_frontend_l4_layers.png`	Livello 4 frontend: layered architecture per coding guidato.
`arch_backend_l4_layers.puml`	`arch_backend_l4_layers.png`	Livello 4 backend: layer applicativi e dipendenze interne.
`arch_domain_class_core.puml`	`arch_domain_class_core.png`	Diagramma classi del core domain (entita e relazioni).
`arch_sequence_checkout.puml`	`arch_sequence_checkout.png`	Diagramma di sequenza del flusso principale di checkout.

Tabella 8: Elenco sorgenti PlantUML e output compilati dell'architettura.

9.1 Convenzioni operative

- ogni file PlantUML deve essere modificabile senza dipendere da asset esterni;
- i nomi dei componenti devono essere allineati ai nomi usati nella sezione 9;
- ogni diagramma include titolo, legenda minima e protocollo principale tra componenti;
- i diagrammi L4 sono orientati alla codifica (classi, package, dipendenze).